

Практикум 317 группы

Задание 2. Градиентные методы обучения линейных моделей. Применение линейных моделей для определения токсичности комментария

ВМК МГУ

Студент: Пескова К.О.

Ноябрь 2025

Содержание

1	Введение	2
2	Теоретическая часть: вывод формул	2
2.1	Градиент функции потерь для бинарной логистической регрессии	2
2.2	Градиент функции потерь для мультиномиальной (многоклассовой) логистической регрессии . .	2
2.3	Сведение мультиномиальной логистической регрессии к бинарной при $K = 2$	3
3	Реализация	3
4	Эксперименты	3
5	Заключение	15

1 Введение

Второе практическое задание направлено на ознакомление с линейными моделями, а также градиентными методами обучения. Работа выполнялась на 3 курсе, в рамках занятий по «Практикуму на ЭВМ».

Целью данной работы является изучение и практическое применение градиентных методов обучения для линейных моделей классификации.

Первая часть работы является теоретической: в ней выводятся формулы градиента функции потерь для задач бинарной и многоклассовой логистической регрессии, а также доказывается, что при количестве классов $= 2$ задача мультиномиальной логистической регрессии сводится к бинарной.

Практическая часть работы посвящена реализации линейного классификатора и исследованию поведения различных вариантов градиентных методов — классического градиентного спуска и стохастического градиентного спуска. Реализованные алгоритмы тестируются на задаче определения токсичности комментариев из раздела обсуждений английской Википедии. Датасет содержит тексты комментариев и метки, указывающие, является ли комментарий токсичным.

Итогом работы является реализация полностью векторизованных функций для вычисления значений и градиентов функции потерь, реализация методов градиентного и стохастического градиентного спуска, а также анализ их эффективности при решении практической задачи классификации текстов. Отчёт подготовлен с использованием системы верстки L^AT_EX.

2 Теоретическая часть: вывод формул

2.1 Градиент функции потерь для бинарной логистической регрессии

Пусть заданы выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$, и параметр $w \in \mathbb{R}^d$.

Модель:

$$p_i = P(y_i = 1 \mid x_i; w) = \sigma(w^\top x_i), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Функция потерь (кросс-энтропия):

$$L(w) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Производная по w для одного объекта:

$$\nabla_w \ell_i(w) = (p_i - y_i) x_i.$$

Следовательно, градиент по всей выборке:

$$\nabla_w L(w) = \sum_{i=1}^n (p_i - y_i) x_i = X^\top (p - y),$$

где $X = [x_1, \dots, x_n]^\top \in \mathbb{R}^{n \times d}$, $p = (p_1, \dots, p_n)^\top$, $y = (y_1, \dots, y_n)^\top$.

$$\nabla_w L(w) = X^\top (p - y).$$

2.2 Градиент функции потерь для мультиномиальной (многоклассовой) логистической регрессии

Пусть число классов K , параметры $W = [w_1, \dots, w_K] \in \mathbb{R}^{d \times K}$.

Модель многоклассовой классификации:

$$p_{ik} = P(y_i = k \mid x_i; W) = \frac{e^{w_k^\top x_i}}{\sum_{j=1}^K e^{w_j^\top x_i}}.$$

Функция потерь:

$$L(W) = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log p_{ik},$$

где $y_{ik} = 1$, если $y_i = k$, и 0 иначе.

Вывод градиента:

Для одного класса k :

$$\nabla_{w_k} L(W) = \sum_{i=1}^n (p_{ik} - y_{ik}) x_i.$$

В матричной форме:

$$\nabla_W L(W) = X^\top (P - Y),$$

где $P = [p_{ik}] \in \mathbb{R}^{n \times K}$, $Y = [y_{ik}] \in \mathbb{R}^{n \times K}$.

$$\nabla_W L(W) = X^\top (P - Y).$$

2.3 Сведение мультиномиальной логистической регрессии к бинарной при $K = 2$

Пусть $K = 2$, тогда

$$p_{i1} = \frac{e^{w_1^\top x_i}}{e^{w_1^\top x_i} + e^{w_2^\top x_i}}, \quad p_{i2} = \frac{e^{w_2^\top x_i}}{e^{w_1^\top x_i} + e^{w_2^\top x_i}}.$$

Тогда

$$p_{i1} = \frac{1}{1 + e^{-(w_1 - w_2)^\top x_i}} = \sigma((w_1 - w_2)^\top x_i).$$

Обозначим $w = w_1 - w_2$, тогда вероятность класса 1:

$$p_i = \sigma(w^\top x_i), \quad p_{i2} = 1 - p_i.$$

Функция потерь:

$$L(W) = - \sum_{i=1}^n [y_{i1} \log p_i + (1 - y_{i1}) \log(1 - p_i)],$$

что совпадает с бинарной логистической регрессией из пункта 1.

Таким образом, при **K = 2** мультиномиальная логистическая регрессия эквивалентна бинарной.

3 Реализация

Описание структуры проекта

В ходе работы над проектом были написаны следующие модули:

1. Модуль `oracles.py`, содержащий реализации функций потерь и их градиентов.
2. Модуль `utils.py` для подсчета градиента произвольного функционала.
3. Модуль `optimization.py` с реализацией методов обыкновенного и стохастического градиентного спуска.

4 Эксперименты

Датасет

Для проведения экспериментов использовался датасет, содержащий комментарии из раздела обсуждений английской Википедии. Каждый комментарий имеет бинарную метку, показывающую, является ли данный текст *токсичным* или *нейтральным*. Таким образом, задача классификации формулируется как задача бинарной логистической регрессии.

Датасет был предоставлен в формате CSV и содержит следующие основные поля:

- `comment_text` — текст комментария;
- `is_toxic` — бинарная метка токсичности.

Эксперимент 1: предварительная обработка текста.

Перед обучением модели была произведена предварительная обработка текста: приведение к нижнему регистру, а также замена всех символов, не являющихся буквами и цифрами, на пробелы. Для реализации были использованы такие функции, как: `str.lower` и `re.sub`.

Эксперимент 2: преобразование в разреженную матрицу.

После предварительной обработки текстов необходимо преобразовать коллекцию комментариев в числовое представление, с которым возможно обучение линейных моделей. Для этого использовалось классическое представление «мешка слов» (Bag-of-Words), в котором каждый документ описывается частотами встречаемости слов из словаря.

Поскольку большинство слов встречается лишь в малом числе документов, полученная матрица X является сильно разреженной. Для эффективного хранения и вычислений она была представлена в формате `scipy.sparse.csr_matrix`, что снижает объём занимаемой памяти.

Для построения такой матрицы использовался класс `CountVectorizer` из библиотеки `sklearn.feature_extraction.text`.

Эксперимент 3: градиентный и стохастический градиентный спуски.

В ходе этого эксперимента были реализованы методы градиентного и стохастического градиентного спусков в модуле `optimization.py` в соответствии с требованиями к заданию. Реализация полностью векторизована и не содержит явных циклов, что обеспечивает высокую эффективность вычислений при работе с разреженными матрицами признаков.

Далее для проверки правильности аналитического выражения для градиента функции потерь была проведена процедура сравнения с численным приближением, реализованным в модуле `utils.py`.

Вектор аналитического и численного градиентов для тестового примера:

$$\text{grad_analytic} = \begin{bmatrix} -0.27911095 \\ -0.28730436 \\ -0.23580188 \end{bmatrix}, \quad \text{grad_numeric} = \begin{bmatrix} -0.27911095 \\ -0.28730436 \\ -0.23580189 \end{bmatrix}.$$

Норма разности между аналитическим и численным градиентами составила примерно:

$$\|\nabla_w^{(\text{analytic})} L - \nabla_w^{(\text{numeric})} L\|_2 \approx 1.05 \times 10^{-8}.$$

Как можно заметить, разница между аналитическим и численным градиентом настолько мала, что ею можно пренебречь — это подтверждает правильность реализации градиента.

Эксперимент 4: исследование поведения градиентного спуска в зависимости от различных параметров.

В данной части работы было исследовано поведение метода градиентного спуска при решении задачи логистической регрессии в зависимости от следующих параметров:

- параметра начального шага обучения `step_alpha`
- параметра затухания шага `step_beta`
- начального приближения

Исследование поведения подразумевало анализ: зависимости значения функции потерь от итерации метода, а также зависимости точности (ассигасу) итерации метода.

Шаг обучения на итерации k задавался в виде

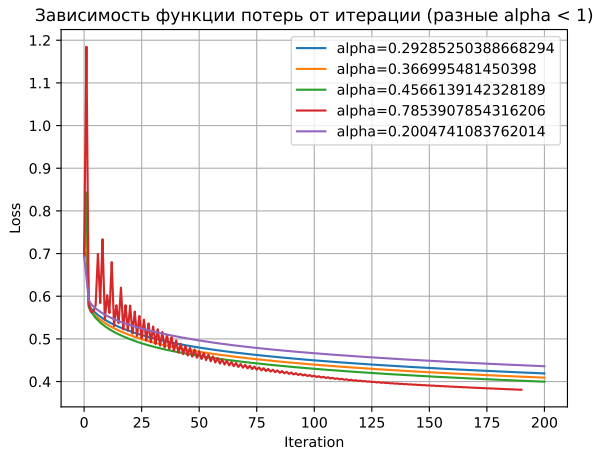
$$\eta_k = \frac{\text{step_alpha}}{(k+1)^{\text{step_beta}}},$$

где `step_alpha` > 0 определяет начальный размер шага, а `step_beta` ≥ 0 — скорость его убывания.

Для этого эксперимента были выбраны по 10 значений `step_alpha` и `step_beta`: 5 из них меньше единицы, и 5 больше. Параметры выбирались случайно, при помощи `np.random`.

Итак, рассмотрим графики зависимости функции потерь от итерации и точности (ассигасу) от итерации при разных `step_alpha`.

Зависимость от step_alpha

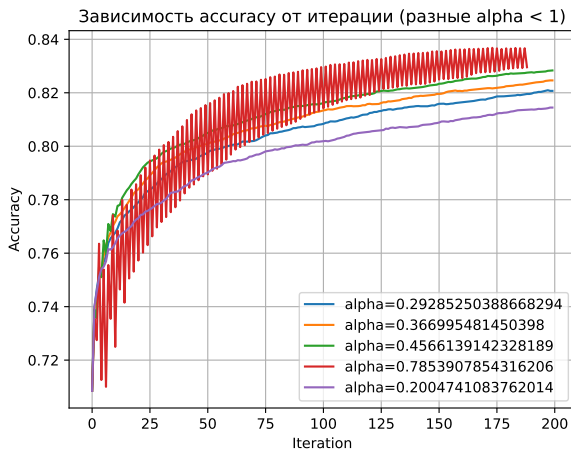


(a) Зависимость функции потерь от итерации при $\alpha < 1$

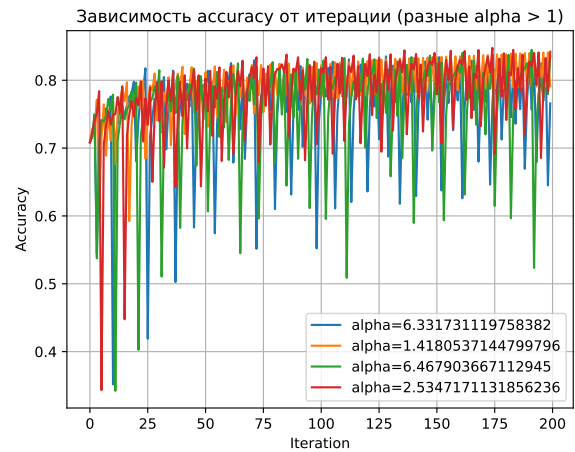


(b) Зависимость функции потерь от итерации при $\alpha > 1$

Рис. 1: Сравнение поведения функции потерь при различных значениях параметра шага α



(a) Зависимость точности (Accuracy) от итерации, $\alpha < 1$



(b) Зависимость точности (Accuracy) от итерации, $\alpha > 1$

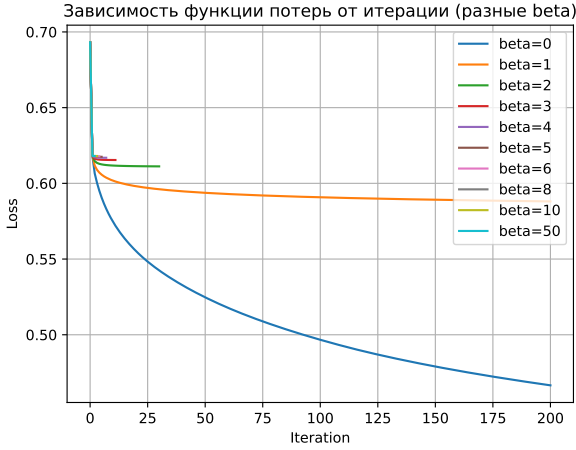
Рис. 2: Сравнение поведения точности модели при различных значениях параметра шага α

Можно отметить, что при всех значениях параметра наблюдается убывание функции потерь, однако скорость сходимости различна. Малые значения **step_alpha** приводят к плавной, но медленной сходимости, в то время как увеличение шага ускоряет процесс на начальных итерациях.

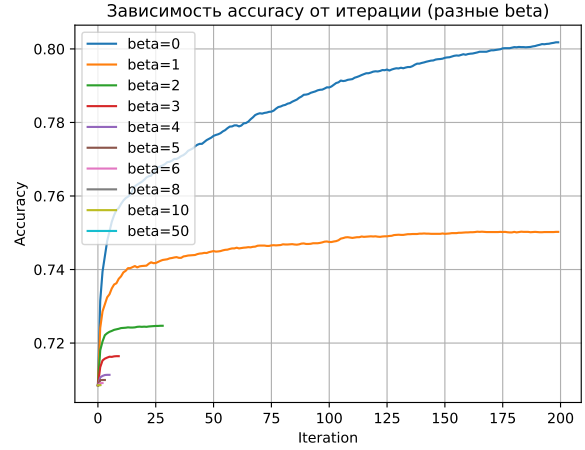
Для **step_alpha** меньше единицы метод демонстрирует устойчивую сходимость: функция потерь монотонно убывает и стремится к минимуму, без существенных колебаний. При увеличении альфа (>1) наблюдается существование колебаний: функция потерь становится нестабильной.

Итого лучшим значением параметра **step_alpha** было выбрано минимальное **alpha**, в нашем случае **best_step_alpha = 0.2**.

Зависимость от step_beta



(a) Зависимость функции потерь от итерации при различных β



(b) Зависимость точности (Accuracy) от итерации при различных β

Рис. 3: Сравнение поведения модели при различных значениях параметра затухания шага β

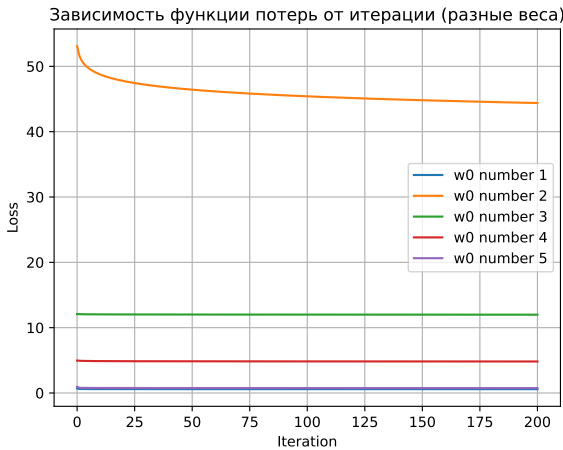
Для исследования зависимости функции потерь и точности от `step_beta` были выбраны следующие значения этого параметра: 0, 1, 2, 3, 4, 5, 6, 8, 10, 50.

Все графики loss-функции, как и следовало ожидать, убывают с числом итераций при всех `beta`, что говорит о сходимости метода. "Быстрее" всего убывает график при `step_beta = 0`. При таком значении шаг обучения η_k линейно зависит от не изменяющегося шага `alpha`, то есть не меняется.

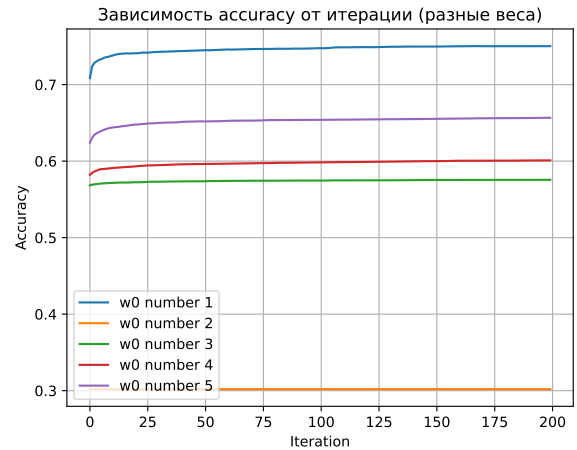
На графике зависимости ассигасы от итерации наблюдается рост, при разных `step_beta` имеющий разную скорость. Здесь также лучше всего себя показывает значение 0.

Итого лучшим параметром `best_step_beta` был выбран 0.

Зависимость от начального приближения.



(a) Зависимость функции потерь от итерации при разных начальных приближениях w_0



(b) Зависимость точности (Accuracy) от итерации при разных начальных приближениях w_0

Рис. 4: Сравнение поведения модели при различных начальных приближениях w_0

В качестве набора начальных приближений, на котором ставился эксперимент были выбраны следующие векторы: нулевой, единичный, вектор из нормального распределения, вектор из равномерного распределения, вектор с малым разбросом.

По графику функции потерь заметно, что убывающим является лишь график при единичном начальном приближении. Остальные же представляют собой почти горизонтальные прямые, но расположенные на разных уровнях. Ниже всего находится график при нулевом начальном приближении.

Точность ассигасы в зависимости от итерации растет очень медленно, а начиная с 50-й итерации графики вновь напоминают почти горизонтальные линии. Однако и здесь выделяется график при нулевом начальном приближении: точность при таком значении w_0 имеет наивысшее значение.

Таким образом, `best_w_0` = нулевому вектору соответствующей размерности.

Эксперимент 5: исследование поведения стохастического градиентного спуска от различных параметров

В 5-м эксперименте предлагалось рассмотреть зависимость поведения стохастического градиентного спуска от:

1. параметр размера шага `step_alpha`
2. параметр размера шага `step_beta`
3. размер подвыборки `batch_size`

По `step_alpha`



(a) Зависимость функции потерь от итерации при $\alpha < 1$ (SGD)

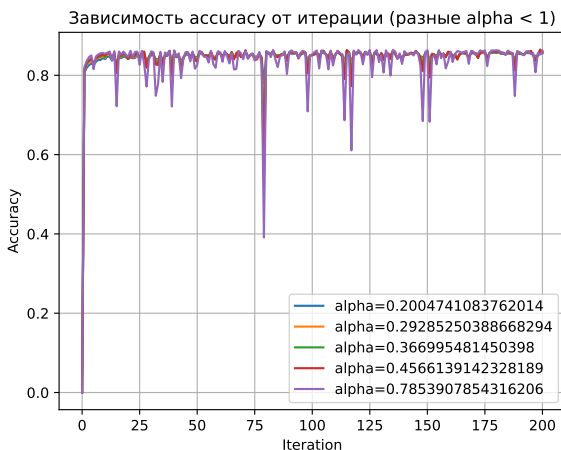


(b) Зависимость функции потерь от итерации при $\alpha > 1$ (SGD)

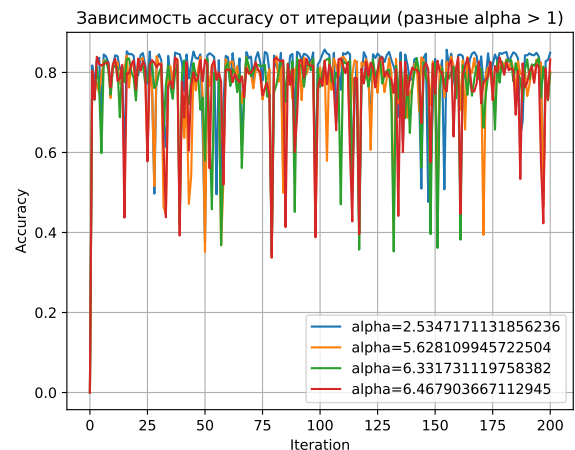
Рис. 5: Сравнение поведения функции потерь стохастического градиентного спуска при различных значениях параметра α

Значения параметра альфа в этом эксперименте были выбраны аналогично предыдущему: 5 меньше единицы, и 5 больше. Параметры выбирались случайно, при помощи `np.random`.

Заметно, что в стохастическом методе графики функции потерь при всех альфа имеют нестабильный характер: видны большие и частые скачки функции. Однако глобально при всех альфа функции потерь все же убывают, пусть и с небольшой скоростью.



(a) Зависимость точности от итерации при $\alpha < 1$ (SGD)



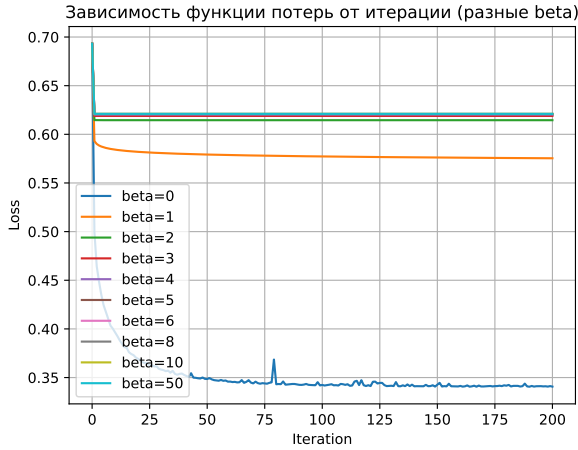
(b) Зависимость точности от итерации при $\alpha > 1$ (SGD)

Рис. 6: Сравнение точности стохастического градиентного спуска при различных значениях параметра α

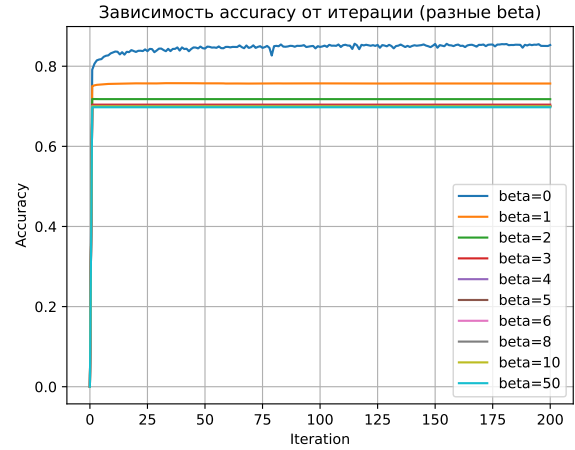
Аналогично с ассигасу: на графики видны частые колебания с большой амплитудой, но глобально точность все же возрастает.

В качестве лучшего параметра `best_alpha_stoch` был выбран наименьший из сгенерированных, то есть **0.2**.

По step_beta



(a) Зависимость функции потерь от итерации при различных значениях параметра β (SGD)



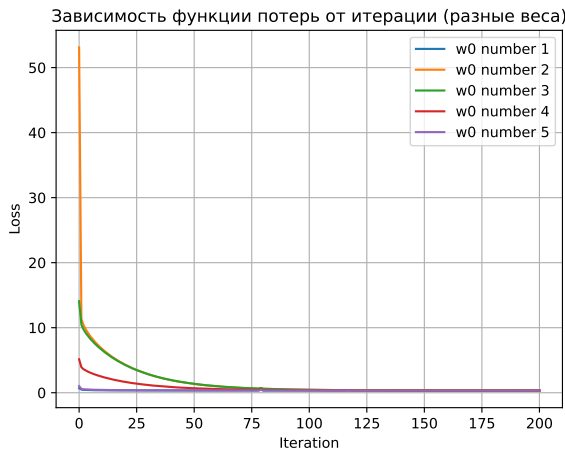
(b) Зависимость точности от итерации при различных значениях параметра β (SGD)

Рис. 7: Поведение стохастического градиентного спуска при изменении параметра β

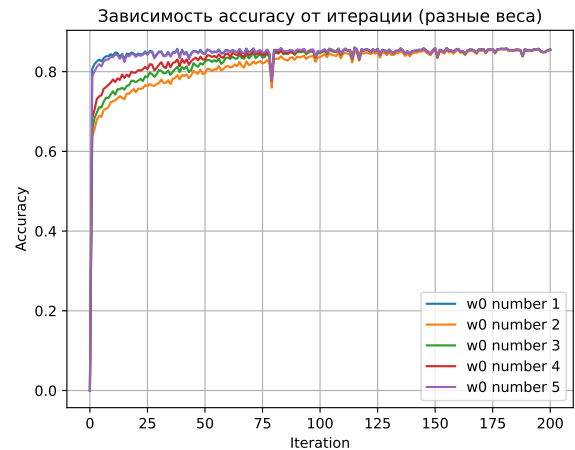
Функции потерь значительно резко падает на начальных итерациях, далее имеет убывающий характер. При значениях, отличных от нуля графики убывают очень медленно, и сама функция на финальной итерации имеет немалое значение: примерно в диапазоне от 0.57 до 0.62. А вот при **step_beta = 0** функция убывает значительно круче и loss имеет минимальное значение среди остальных **beta**: меньше 0.35.

График accuracy имеет схожее поведение, только зеркальное по горизонтали: наибольшее значение принимается при **best_beta_stoch = 0**,

По начальному приближению



(a) Зависимость функции потерь от итерации при различных начальных приближениях w_0 (SGD)



(b) Зависимость точности от итерации при различных начальных приближениях w_0 (SGD)

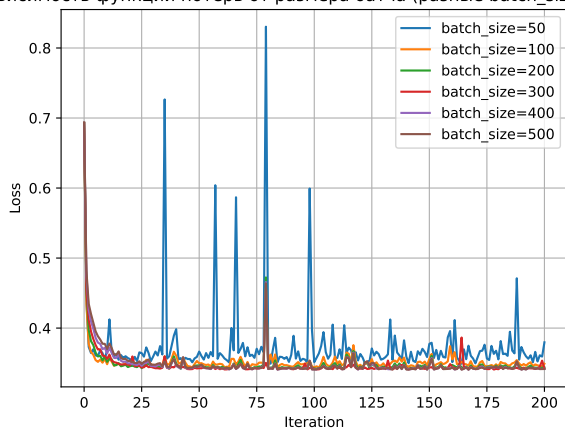
Рис. 8: Поведение стохастического градиентного спуска при разных начальных приближениях w_0

В ходе обучения при пяти различных начальных приближениях (аналогичных эксперименту 4) наблюдается быстрая сходимость функции потерь на первых 50-80 итерациях и последующее плато. Разброс кривых на ранних итерациях существенен, но к 150-200-й итерации все запуски сходятся к близким значениям точности ($\approx 0.86-0.87$).

Лучшим начальным приближением **w0_best_stoch** было также выбрано нулевое.

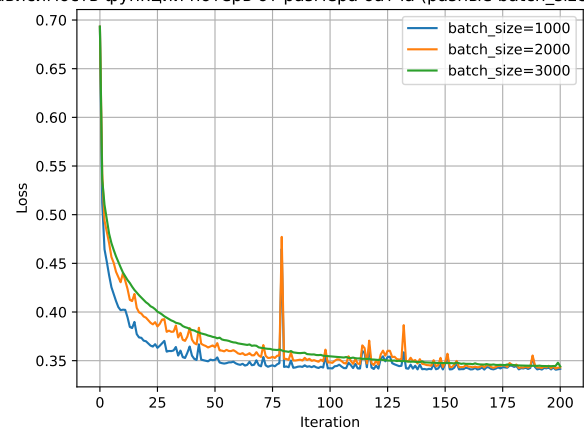
По batch_size

Зависимость функции потерь от размера батча (разные batch_size < 100)



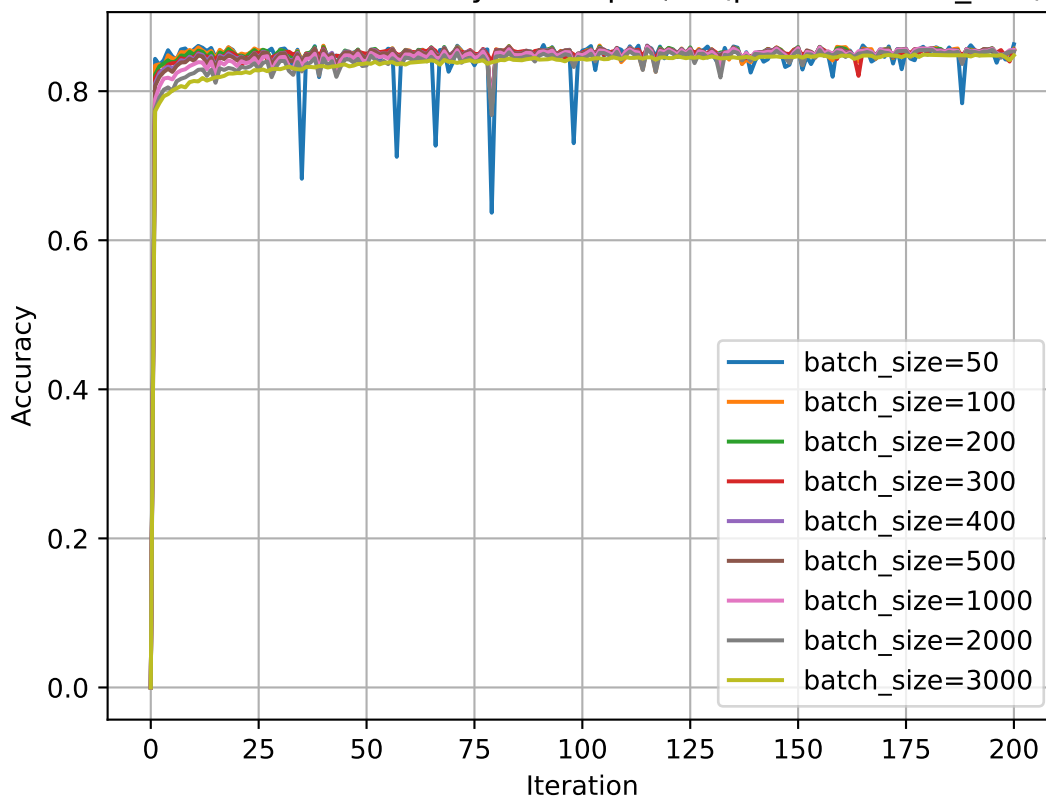
(a) Зависимость функции потерь от итерации при малых размерах батча (batch_size < 1000)

ависимость функции потерь от размера батча (разные batch_size >= 100)



(b) Зависимость функции потерь от итерации при больших размерах батча (batch_size ≥ 1000)

Зависимость accuracy от итерации (разные batch_size)



(c) Зависимость точности от итерации при различных размерах батча (SGD)

Рис. 9: Поведение стохастического градиентного спуска при различных размерах батча

При малых размерах батча loss имеет примерно одинаковый итоговый уровень, не смотря на частые колебания. При батчах крупнее траектории становятся более гладкими и стабильными.

Все варианты batch_size быстро выходят в плато на графике accuracy, но при самом большом батче, размера 3000, линия становится самой гладкой.

Вывод: выбираем **best_batch_size = 3000**.

Эксперимент 6: сравнение методов градиентного и стохастического градиентного спуска.

В ходе данного эксперимента необходимо было сравнить поведение функции потерь, точности, а также времени выполнения в зависимости от итераций при разных методах.

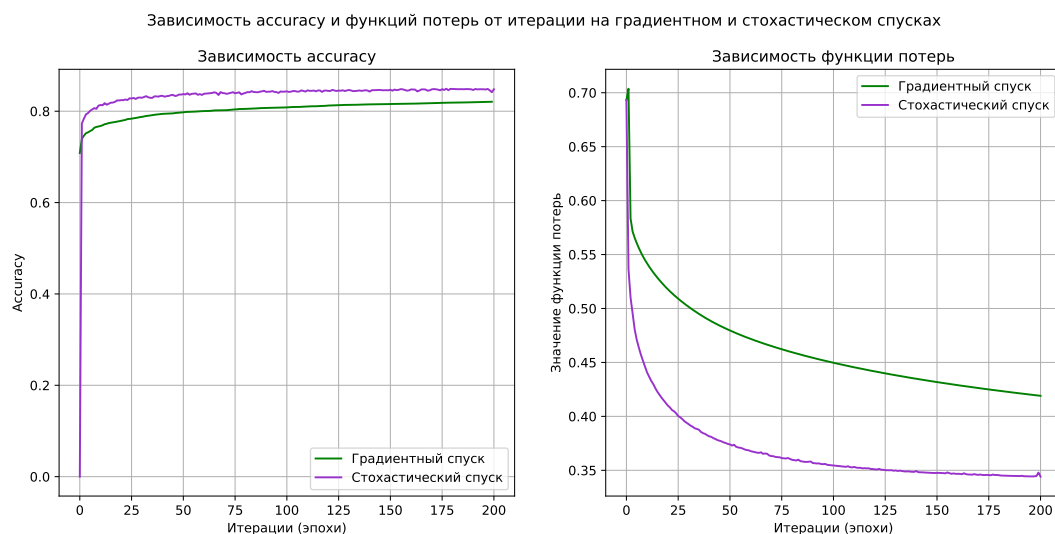


Рис. 10: Сравнение зависимости ассигасы и функции потерь от итерации при градиентном и стохастическом спусках

Стохастический градиентный спуск сходится быстрее, чем обычный градиентный, поскольку на каждой итерации использует меньше данных и чаще обновляет веса. Благодаря стохастическому шуму SGD также избегает локальных минимумов и достигает более низкого значения функции потерь, обеспечивая лучшее качество на тестовой выборке.

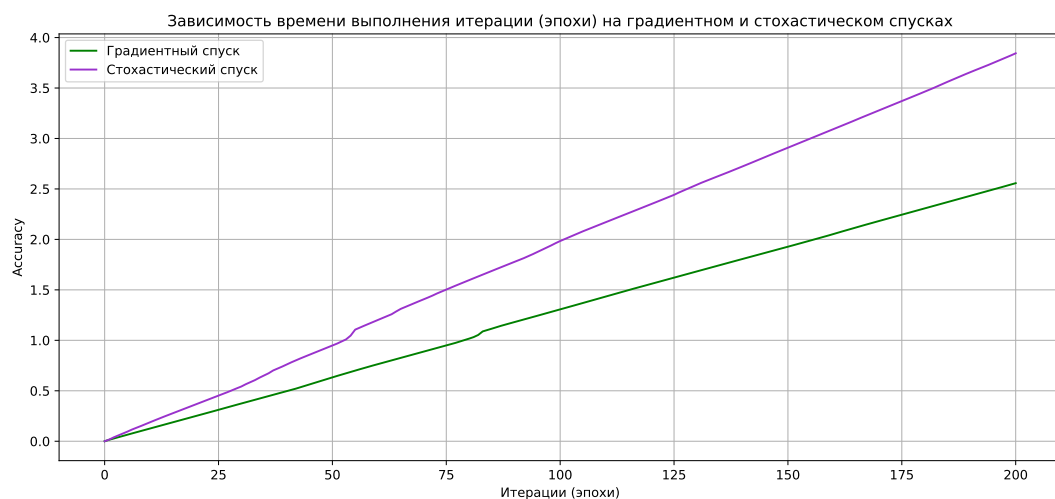


Рис. 11: Зависимость времени выполнения от итерации при градиентном и стохастическом спусках

На графике зависимости времени от итерации видно, что время выполнения стохастического градиентного спуска больше, чем у обычного.

Это связано с тем, что одна эпоха SGD содержит множество обновлений весов (по мини-батчам), а также с дополнительными вычислительными затратами при частом сохранении метрик. Однако, несмотря на большее время на эпоху, стохастический спуск достигает лучшего качества и меньшей функции потерь за меньшее количество итераций, то есть сходится быстрее.

Эксперимент 7: лемматизация.

В данной части работы к коллекции был применен алгоритм лемматизации (процесс приведения слова к его лемме с учётом части речи и значения), а также были удалены стоп-слова. Для этого использовалась библиотека

nlk.

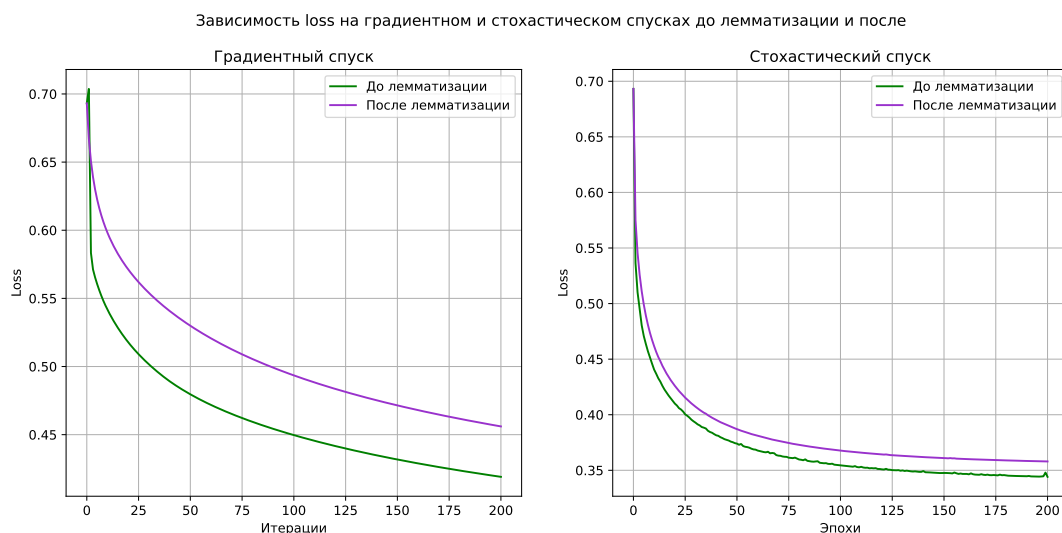


Рис. 12: Сравнение функции потерь до и после лемматизации для градиентного и стохастического спусков

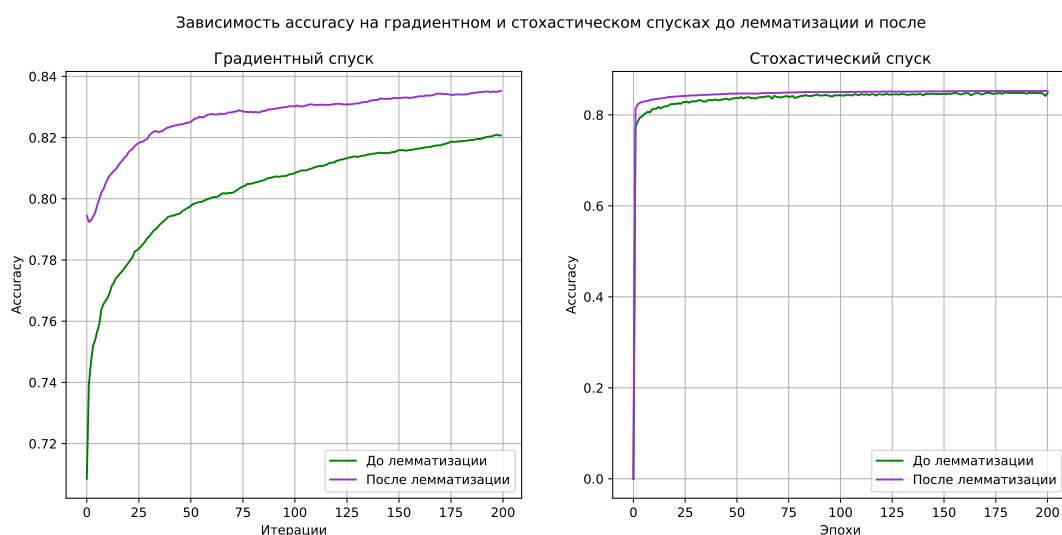


Рис. 13: Сравнение точности (ассигасы) до и после лемматизации для градиентного и стохастического спусков

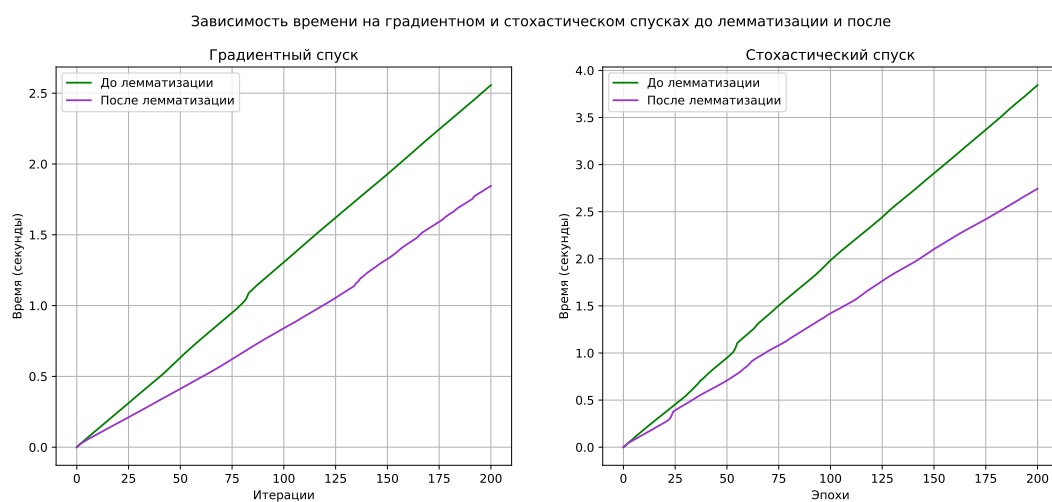


Рис. 14: Сравнение времени выполнения итераций до и после лемматизации для градиентного и стохастического спусков

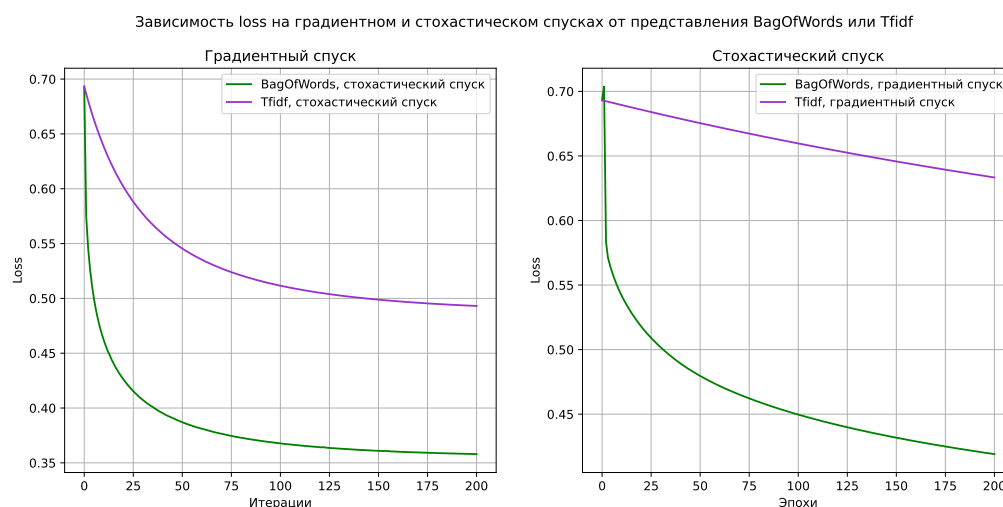
Такая предобработка повысила точность классификации как в случае стохастического градиентного спуска, так и в случае обычного, что объясняется уменьшением шума в данных и размера словаря. Также замечен спад времени работы методов, вызванный уменьшением матриц обучающей и тестовой выборки. При этом значение функции потерь немного возросло, что объясняется тем, что модель стала менее чувствительной к частным вариантам словоформ, но более устойчивой и обобщающей. Таким образом, лемматизация улучшила качество классификации и эффективность обучения при незначительном увеличении значения loss.

Эксперимент 8: исследование поведения алгоритма в зависимости от параметров лемматизации.

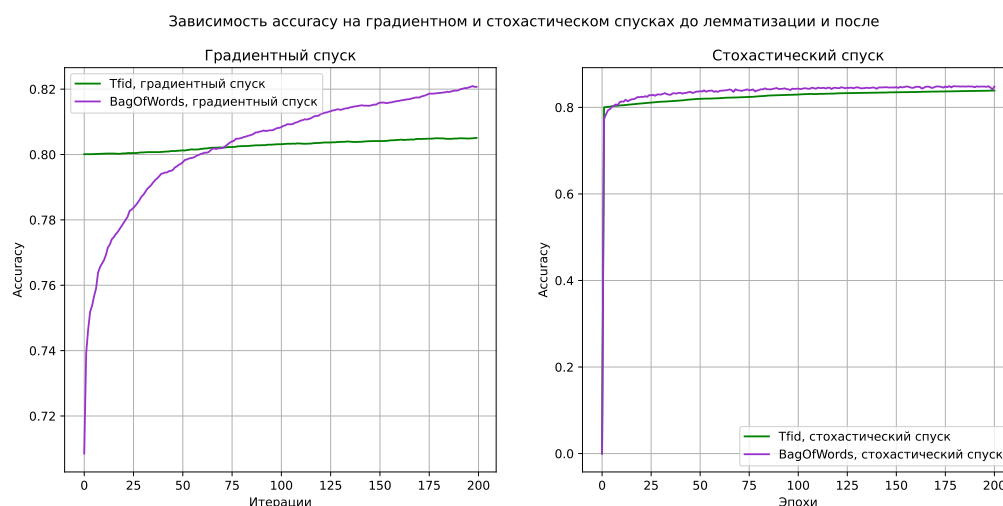
Цель эксперимента: проверить, как на качество классификации, время обучения и размер признакового пространства влияют:

- выбор текстового представления (Bag-of-Words или TF-IDF),
- параметры `min_df` и `max_df` у векторизаторов.

Bag-of-Words или TF-IDF



(a) Зависимость функции потерь от представления (Bag-of-Words / TF-IDF)



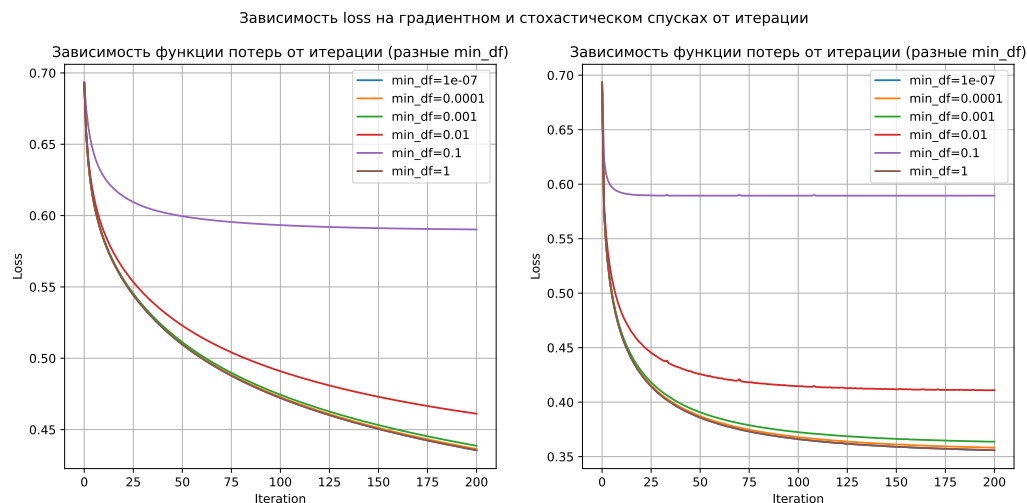
(b) Зависимость точности (ассигуры) от представления (Bag-of-Words / TF-IDF)

Рис. 15: Сравнение представлений Bag-of-Words и TF-IDF на градиентном и стохастическом спусках

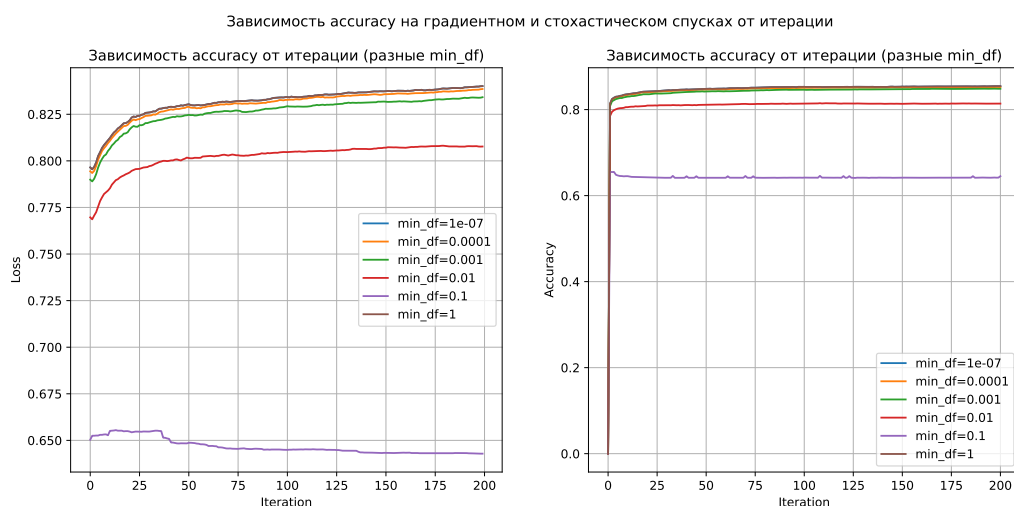
Сравнение показало, что представление Bag-of-Words обеспечивает более быструю сходимость функции потерь и несколько более высокую точность по сравнению с TF-IDF. При этом TF-IDF характеризуется более плавной сходимостью и устойчивостью обучения, что указывает на его лучшую способность к обобщению данных.

По min_df и max_df

Параметр `min_df` задаёт минимальную долю документов, в которых слово должно встретиться, чтобы попасть в словарь. На графике видно, что изменение параметра `min_df` оказывает значительное влияние на результаты: при малых значениях (от $1e-6$ до $1e-3$) сохраняется больше редких слов, а значит точность выше и сходимость быстрее.

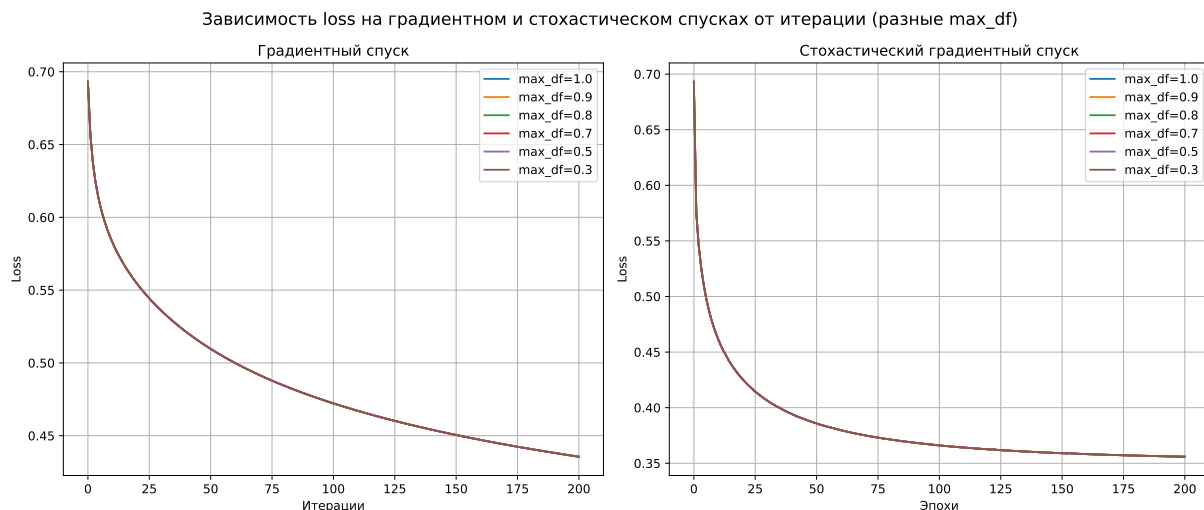


(а) Зависимость функции потерь от параметра `min_df`

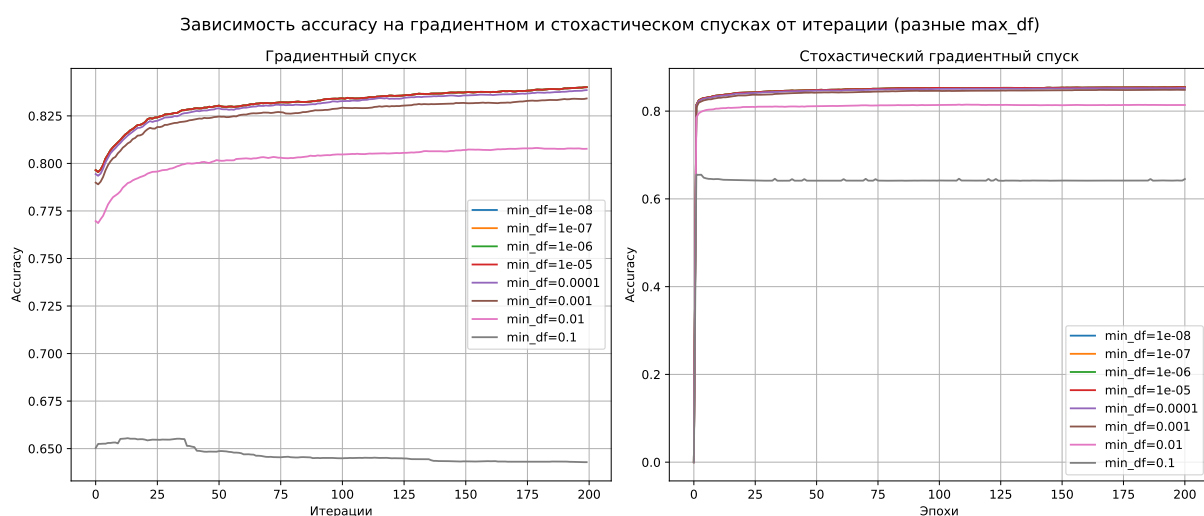


(б) Зависимость точности (ассигасы) от параметра `min_df`

Рис. 16: Влияние параметра `min_df` на качество и сходимость градиентного и стохастического спусков



(a) Зависимость функции потерь от параметра `max_df`



(b) Зависимость точности (ассигуры) от параметра `max_df`

Рис. 17: Влияние параметра `max_df` на качество и сходимость градиентного и стохастического спусков

Слова, встречающиеся в доле документов, превышающей `max_df`, исключаются из модели. Как можно видеть, этот параметр напротив, не влияет на качество и точность модели, это объясняется предварительным удалением стоп-слов.

Эксперимент 9: анализ ошибок лучшего алгоритма.

Пришло время проанализировать ошибки лучшего алгоритма. Исходя из проведенных экспериментов, выбираем метод стохастического градиентного спуска, с параметрами `best_step_alpha`, `best_step_beta`, `best_batch_size`, `best_w_0`. Выбираем коллекцию с примененным алгоритмом лемматизации, удаленными стоп-словами, а также в представлении `BagOfWords`.

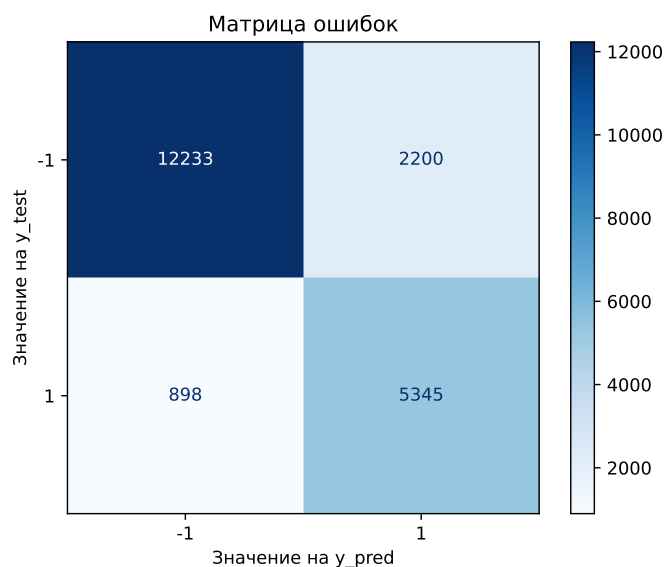


Рис. 18: Матрица ошибок для лучшего алгоритма (SGD, лемматизированные данные, Bag-of-Words)

Проанализируем матрицу ошибок: видно, что модель хорошо справляется с классификацией большей части данных, заметно также, что почти в два раза лучше модель классифицирует негативные метки. Однако наблюдаются и ненулевое число ложноположительных и ложноотрицательных показаний: **2200** и **898** соответственно.

5 Заключение

В рамках задания были реализованы методы градиентного и стохастического градиентного спусков, а также проведены исследования на датасете, содержащем комментарии из раздела обсуждений английской Википедии. Эксперименты включали в себя рассмотрение методов при различных параметрах обучения, а также способах представления данных (BagOfWords, Tfidf).

Результаты показали, что метод стохастического градиентного спуска быстрее сходится и достигает более высокой точности, по сравнению с обычным градиентным спуском.

Использование лемматизации продемонстрировало повышение качества классификации и снижение времени обучения. В сравнении представлений лучше себя показало BagOfWords.

Итого оптимальным по качеству был выбран метод стохастического градиентного спуска, с параметрами: **best_alpha_stoch = 0.2**, **best_beta_stoch = 0**, **best_batch_size = 3000**, использованный на лемматизированных данных без стоп-слов, имеющих представление BagOfWords.